

Before You Start

1	Binaries	Download the pre-built spiritbun TurboQuant package. It contains llama-server.exe, llama-bench.exe, llama-cli.exe and all ggml-*.dll / llama.dll files.
2	NVIDIA GPU	Install CUDA Toolkit 12.x or 13.x (nvidia.com/cuda). Copy the matching cuBLAS DLLs (cublas64_12.dll + cublasLt64_12.dll for CUDA 12, or cublas64_13.dll + cublasLt64_13.dll for CUDA 13) into the llama-server folder. These files are not bundled.
3	AMD GPU	Install ROCm 6.x (rocm.docs.amd.com). Use the HIP/ROCm build variant of llama-server. No extra DLLs needed — ROCm runtime ships its own libraries.
4	macOS (Apple Silicon)	Metal is built-in — no extra drivers or DLLs required. Use the macOS build variant. Works on M1/M2/M3/M4.
5	First model	Download a GGUF model from Hugging Face. Good starter: meta-llama-3.1-8b-instruct-q4_k_m.gguf (~4.6 GB). Bigger / better: Qwen3.5-9B-UD-Q4_K_XL.gguf (~5.6 GB). Place the file anywhere, then set the folder in QLauncher → Paths.

⚠ **CUDA version compatibility:**

CUDA 12.4–12.8: fully tested, recommended. CUDA 12.8+ required for RTX 5090 (sm_120). CUDA 13.2: experimental, may work. CUDA 13.1: DO NOT USE — known MMQ kernel segfault that causes crashes during inference. If you need CUDA 13.x, use 13.2 or newer.



Where to find the CUDA DLLs (Windows):

Start → search for “NVIDIA Control Panel” → Help → System Information shows your driver version. Then download the matching CUDA Toolkit. The two required DLLs are in the bin\ subfolder. Alternatively, copy them from any machine that already has CUDA 12.x installed.

What is TurboQuant QLauncher?

TurboQuant QLauncher is a lightweight desktop GUI for managing llama-server with TurboQuant KV-Cache compression. It lets you launch large language models on any NVIDIA (or AMD) GPU with a single double-click, switch between models instantly, run benchmarks, and monitor VRAM usage — all without touching the command line.



Why TurboQuant?

Standard llama.cpp stores the KV-cache in full float16 precision. TurboQuant compresses it using Walsh-Hadamard rotation (ICLR 2026, Google Research), achieving 4–5× smaller KV-cache with minimal quality loss. This means longer context, more VRAM headroom — or fitting a bigger model.

Key Features

⚡	One-click Launch	Double-click any model row to start llama-server on that GPU. Double-click again to stop.
🔦 🔦	Live VRAM Bars	Braille-style bars show effective VRAM usage — updated live when you switch KV config.
🔦 🔦	KV-aware Colors	Green = fits comfortably. Amber = tight. Red = overflow. Changes with compression selection.
🔦 🔦	Benchmarking	Built-in llama-bench: single config or Bench All (all 6 KV configs in one run). f16 baseline + deltas, saved as Markdown.
⌚	Loading vs Running	Status shows ⌚ Loading while model loads into VRAM, then switches to ● Running (click to open browser).
🔦 🔦	Persistent Config	Window position, splitter, KV selection, port and all settings are saved automatically on exit.
🔦 🔦	Auto-scan Models	Scans your model directory for *.gguf files and shows size, fit, and compression estimate per GPU.
🔦 🔦	Layer-Adaptive	Keep critical layers (first/last) at q8_0 for best quality. Modes 1–7 via LA spinner, passed as TURBO_LAYER_ADAPTIVE env var.
🔦	Bench All	Matrix benchmark: all KV configs × all LA modes in one run. Stop anytime between runs.

		
 	Layer-Adaptive	Keep critical layers (first/last) at q8_0 for best quality. Modes 1–7 via LA spinner, passed as TURBO_LAYER_ADAPTIVE env var.

Controls at a Glance

Double-click row	Launch server with selected model on that GPU (or stop if already running)
Single click	Select a model / GPU row without launching
↑ ↓ / ← →	Navigate models (up/down) and GPUs within a model (left/right)
Enter	Toggle server for currently selected row
Escape	Stop the running server
Click status label	Open <code>http://localhost:{port}</code> in your browser (only when Running)
Drag splitter	Resize model list vs. log area
LA: off/1/5/7	Select Layer-Adaptive mode (uniform / first4+last4 / aggressive / boundary-V)
☐ Bench All	Activates all KV + LA buttons (blue). Click individual buttons to deselect. Press Run to start.
Run / Stop	Start benchmark with selected configs. Stop cancels after current run.
LA buttons (off/1/5/7)	Set Layer-Adaptive mode. In Bench All: toggle individual modes on/off
☐ Bench All	Activates all KV + LA buttons (blue). Click buttons to deselect. Press Run to start.
Run / Stop	Run: start benchmark with selected KV × LA matrix. Stop: cancel after current run.

KV-Cache Configurations



Tip:

The VRAM bars and ✓/✗ indicators update live as you click KV buttons. Watch a model that shows amber on your GPU turn green when you select turbo3/turbo3!

Benchmarking & Documentation

QLauncher integrates llama-bench directly, so you can systematically benchmark every KV-Cache configuration without leaving the GUI. Results are saved as Markdown tables — ready for GitHub posts, internal documentation, or comparison across builds and hardware.

☐ Benchmark	Single-config mode. Select a model + GPU row, check Benchmark, then double-click, Enter, or Run. Runs f16 baseline + the currently selected KV config. Delta (%) is calculated automatically.
☐ Bench All	Full-matrix mode. Check Bench All → all KV + LA buttons turn blue. Click individual buttons to deselect configs you don't need. Press Run to start KV × LA matrix (e.g. 6 KV × 4 LA = 24 runs). Stop cancels after the current run.
 Save Results	After a benchmark completes, the  Save Results button turns green. Click to append the results to TurboQuant_Benchmark.md next to the launcher. Each run is a timestamped Markdown table with model name, GPU, KV config, and LA mode.
Benchmark.md	The file accumulates results over time — building a persistent log across CUDA versions, builds, models, and GPUs. Ideal for tracking regressions, comparing spiritbuun vs. thetom builds, or preparing community posts.



Testing workflow for new builds:

1) Build new binaries → Update Binaries. 2) Select your reference model (e.g. Llama 3.1 8B Q4_K_M). 3) Check Bench All, deselect LA modes you don't need. 4) Press Run → wait. 5)  Save. 6) Repeat on second GPU. The resulting Markdown tables are ready to paste into Discussion #20969.

Open Source Projects & Acknowledgements

	spiritbuun / llama.cpp TurboQuant CUDA implementation · this build	github.com/spiritbuun/llama-cpp-turboquant-cuda
---	--	---

★	thetom42 / llama.cpp Alternative TurboQuant build	github.com/TheTom/turboquant_plus
★	ggml-org / llama.cpp Upstream llama.cpp project	github.com/ggml-org/llama.cpp
💡 💡	Community Discussion Benchmark findings · seanrasch · AmesianX · primoco et al.	github.com/ggml-org/llama.cpp/discussions/20969



TurboQuant paper:

Zandieh et al., “TurboQuant: KV Cache Compression via Walsh–Hadamard Rotation”, ICLR 2026, Google Research. This tool is an independent community project, not affiliated with Google.

v0.41 · Layer-adaptive modes · Hover fix · Dynamic benchmark columns

Voraussetzungen

1 Binaries	Lade das vorkompilierte spiritbuun TurboQuant-Paket herunter. Es enthält llama-server.exe, llama-bench.exe, llama-cli.exe und alle ggml-*.dll / llama.dll-Dateien.
2 NVIDIA-GPU	Installiere CUDA Toolkit 12.x oder 13.x (nvidia.com/cuda). Kopiere die passenden cuBLAS-DLLs (cublas64_12.dll + cublasLt64_12.dll für CUDA 12 bzw. cublas64_13.dll + cublasLt64_13.dll für CUDA 13) in den llama-server-Ordner. Diese Dateien sind nicht im Paket enthalten.
3 AMD-GPU	Installiere ROCm 6.x (rocm.docs.amd.com). Verwende die HIP/ROCm-Build-Variante von llama-server. Keine extra DLLs nötig — die ROCm-Runtime liefert eigene Bibliotheken mit.
4 macOS (Apple Silicon)	Metal ist eingebaut — keine zusätzlichen Treiber oder DLLs erforderlich. Verwende die macOS-Build-Variante. Läuft auf M1/M2/M3/M4.
5 Erstes Modell	Lade ein GGUF-Modell von Hugging Face herunter. Guter Einstieg: meta-llama-3.1-8b-instruct-q4_k_m.gguf (~4,6 GB). Größer / besser: Qwen3.5-9B-UD-Q4_K_XL.gguf (~5,6 GB). Datei beliebig ablegen, Ordner dann in QLauncher → Paths eintragen.

⚠ CUDA-Versionskompatibilität:

CUDA 12.4–12.8: vollständig getestet, empfohlen. CUDA 12.8+ erforderlich für RTX 5090 (sm_120). CUDA 13.2: experimentell, kann funktionieren. CUDA 13.1: NICHT VERWENDEN — bekannter MMQ-Kernel-Segfault, der Abstürze während der Inferenz verursacht. Falls CUDA 13.x benötigt, mindestens 13.2 oder neuer verwenden.

💡 Wo finde ich die CUDA-DLLs? (Windows)

Start → „NVIDIA Control Panel“ suchen → Hilfe → Systeminformationen zeigt die Treiberversion. Passenden CUDA Toolkit herunterladen. Die zwei benötigten DLLs liegen im Unterordner bin\.

Alternativ: von einem Rechner kopieren, auf dem CUDA 12.x bereits installiert ist.

Was ist TurboQuant QLauncher?

TurboQuant QLauncher ist eine schlanke Desktop-GUI zum Verwalten von llama-server mit TurboQuant-KV-Cache-Komprimierung. Per Doppelklick startest du ein Sprachmodell auf einer beliebigen NVIDIA- (oder AMD-)GPU, wechselst blitzschnell zwischen Modellen, führst Benchmarks durch und überwachst den VRAM-Verbrauch — ohne Kommandozeile.

💡 Warum TurboQuant?

Standard-llama.cpp speichert den KV-Cache in voller float16-Genauigkeit. TurboQuant komprimiert ihn mittels Walsh-Hadamard-Rotation (ICLR 2026, Google Research) auf 4–5× kleinere Größe — bei minimalem Qualitätsverlust. Das Ergebnis: längere Kontextfenster, mehr VRAM-Spielraum oder Platz für größere Modelle.

Hauptfunktionen

⚡	Ein-Klick-Start	Doppelklick auf eine Modellzeile startet llama-server auf dieser GPU. Nochmal doppelklicken stoppt.
🔦	Live-VRAM-Balken	Braille-Balken zeigen den effektiven VRAM-Verbrauch — werden live aktualisiert beim KV-Wechsel.
🎨	KV-bewusste Farben	Grün = passt bequem. Gelb = knapp. Rot = Überlauf. Ändert sich mit der Kompressionsauswahl.
🏎️	Benchmarking	Integriertes llama-bench: Einzelconfig oder Bench All (alle 6 KV-Configs in einem Durchlauf). f16-Baseline + Deltas, gespeichert als Markdown.
🕒	Laden vs. Laufend	Status zeigt 🕒 Laden während das Modell in den VRAM geladen wird, wechselt dann zu ● Laufend (Klick öffnet Browser).
💡	Persistente Config	Fensterposition, Splitter, KV-Auswahl, Port und alle Einstellungen werden beim Schließen automatisch gespeichert.
🔍	Auto-Scan Modelle	Scannt das Modellverzeichnis nach *.gguf-Dateien und zeigt Größe, Fit und Kompressionsschätzung pro GPU.

 Layer-Adaptive	Kritische Layer (erste/letzte) bei q8_0 halten für beste Qualität. Modi 1-7 per LA-Spinner, übergeben als TURBO_LAYER_ADAPTIVE.
 Bench All	Matrix-Benchmark: alle KV-Configs × alle LA-Modi in einem Lauf. Jederzeit zwischen Runs stoppbar.
 Layer-Adaptive	Kritische Layer (erste/letzte) bleiben bei q8_0 für beste Qualität. Modi 1-7 über LA-Spinner, wird als TURBO_LAYER_ADAPTIVE ENV-Variable gesetzt.

Steuerung auf einen Blick

Doppelklick Zeile	Server mit ausgewähltem Modell auf dieser GPU starten (oder stoppen, wenn bereits aktiv)
Einfachklick	Modell / GPU-Zeile auswählen, ohne zu starten
↑ ↓ / ← →	Modelle navigieren (hoch/runter) und GPUs innerhalb eines Modells (links/rechts)
Enter	Server für aktuell ausgewählte Zeile umschalten
Escape	Laufenden Server stoppen
Klick Statuslabel	http://localhost:{port} im Browser öffnen (nur wenn Laufend)
Splitter ziehen	Modellliste vs. Log-Bereich vergrößern/verkleinern
LA: off/1/5/7	Layer-Adaptive-Modus wählen (uniform / erste4+letzte4 / aggressiv / Boundary-V)
☐ Bench All	Aktiviert alle KV + LA Buttons (blau). Einzelne Buttons anklicken zum Abwählen. Run startet.
Run / Stop	Benchmark starten / nach aktuellem Run abbrechen.
LA-Buttons (off/1/5/7)	Layer-Adaptive-Modus setzen. Bei Bench All: einzelne Modi an/aus toggeln
☐ Bench All	Aktiviert alle KV- + LA-Buttons (blau). Buttons klicken zum Abwählen. Run drücken zum Starten.
Run / Stop	Run: Benchmark mit ausgewählter KV × LA-Matrix starten. Stop: nach aktuellem Run abbrechen.

KV-Cache-Konfigurationen

Tipp:

Die VRAM-Balken und ✓/X-Anzeigen aktualisieren sich live beim Klick auf KV-Buttons. Beobachte, wie ein Modell, das auf deiner GPU Gelb anzeigt, beim Wechsel auf turbo3/turbo3 Grün wird!

Benchmarking & Dokumentation

QLauncher integriert llama-bench direkt, sodass alle KV-Cache-Konfigurationen systematisch getestet werden können — ohne die GUI zu verlassen. Ergebnisse werden als Markdown-Tabellen gespeichert, fertig für GitHub-Posts, interne Dokumentation oder Vergleiche zwischen Builds und Hardware.

☐ Benchmark	Einzelkonfiguration. Modell + GPU-Zeile wählen, Benchmark ankreuzen, dann Doppelklick, Enter oder Run. Führt f16-Baseline + die aktuell gewählte KV-Config aus. Delta (%) wird automatisch berechnet.
☐ Bench All	Vollmatrix-Modus. Bench All ankreuzen → alle KV- + LA-Buttons werden blau. Einzelne Buttons klicken zum Abwählen. Run drücken startet KV × LA-Matrix (z.B. 6 KV × 4 LA = 24 Runs). Stop bricht nach dem aktuellen Run ab.
 Speichern	Nach Abschluss wird der  Save Results-Button grün. Klick hängt die Ergebnisse an TurboQuant_Benchmark.md an. Jeder Run ist eine Markdown-Tabelle mit Zeitstempel, Modellname, GPU, KV-Config und LA-Modus.
Benchmark.md	Die Datei sammelt Ergebnisse über die Zeit — ein persistentes Log über CUDA-Versionen, Builds, Modelle und GPUs. Ideal für Regressionstests, Vergleich spiritbuun vs. thetom, oder Vorbereitung von Community-Posts.

Test-Workflow für neue Builds:

1) Neue Binaries bauen → Update Binaries. 2) Referenzmodell wählen (z.B. Llama 3.1 8B Q4_K_M). 3) Bench All ankreuzen, unnötige LA-Modi abwählen. 4) Run drücken → warten. 5)  Speichern. 6) Auf zweiter GPU wiederholen. Die Markdown-Tabellen sind sofort bereit für Discussion #20969.

Open-Source-Projekte & Danksagungen

★	spiritbuun / llama.cpp TurboQuant-CUDA-Implementierung · dieser Build	github.com/spiritbuun/llama-cpp-turboquant-cuda
★	thetom42 / llama.cpp Alternativer TurboQuant-Build	github.com/TheTom/turboquant_plus
★	ggml-org / llama.cpp Upstream-llama.cpp-Projekt	github.com/ggml-org/llama.cpp
💡 💡	Community-Diskussion Benchmark-Ergebnisse · seanrasch · AmesianX · primoco et al.	github.com/ggml-org/llama.cpp/discussions/20969

 **TurboQuant-Paper:**
Zandieh et al., „TurboQuant: KV Cache Compression via Walsh-Hadamard Rotation“, ICLR 2026, Google Research. Dieses Tool ist ein unabhängiges Community-Projekt und nicht mit Google verbunden.

v0.41 · Layer-adaptive Modi · Hover-Fix · Dynamische Benchmark-Spalten